

Lecture 6

Old-school positioning

- Display
 - block, inline, inline-block, none
- Position
 - static, relative, absolute, fixed, sticky
 - + top, bottom, left, right
- CSS normal flow vs. float

New flexible design

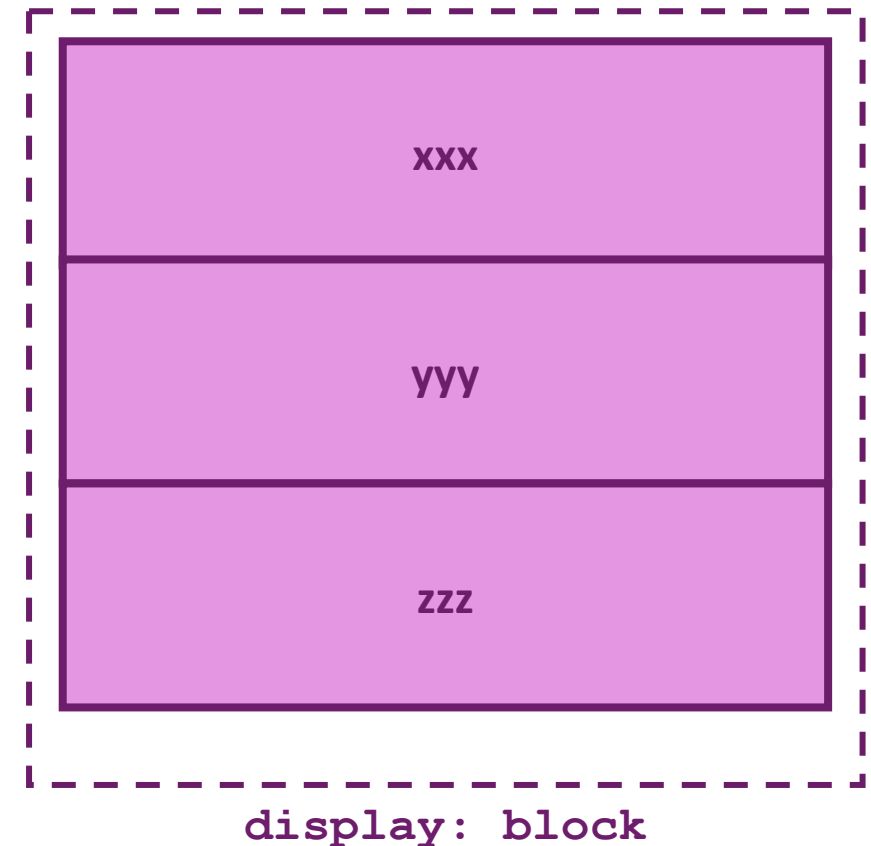
CSS Flexbox (1D)

CSS Grid (2D)

```
<div>xxx</div>  
<div>zzz</div>  
<div>yyy</div>  
...
```

CSS property `display: block`

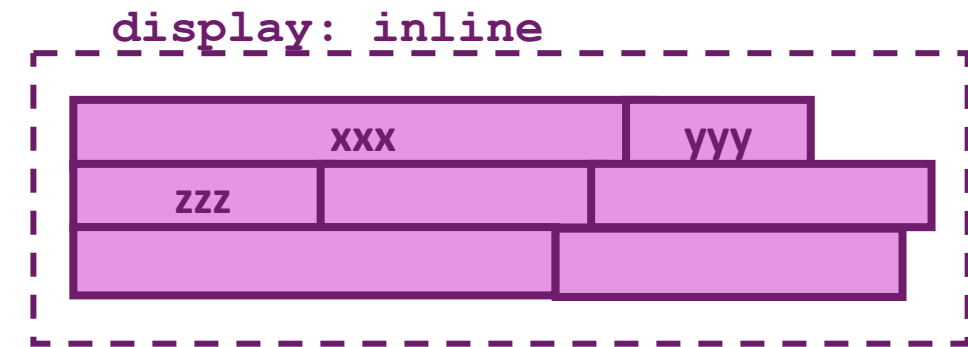
- Elements have default values for the property
 - can be changed with a CSS rule
- Block level elements
`<div>`, `<p>`, `<ol>`, `<li>`, ...
 - default value, `display: block`
 - takes the whole width of the parent
 - `margin`, `padding`, `width`, `height` settable



```
<span>xxx</span>  
<span>zzz</span>  
<span>yyy</span>  
...
```

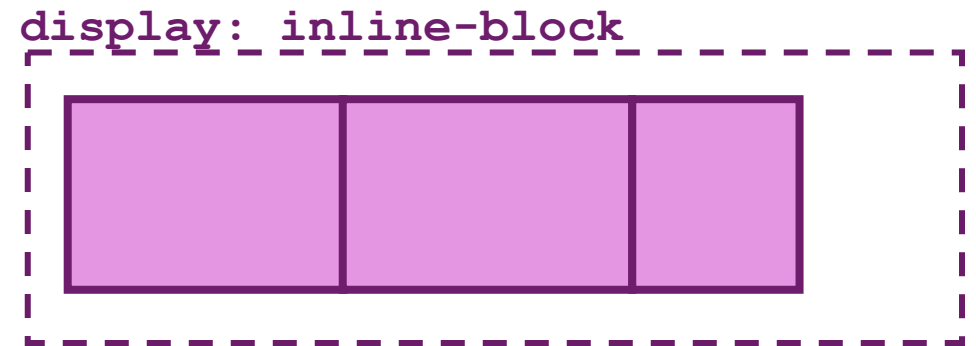
CSS property `display: inline`

- Inline elements include
`<span>`, `<strong>`, `<em>`, ...
- default value, `display: inline`
- **width** and **height** specified by content
 - **padding**/**margin** ineffective on vertical direction
- in line with other inline elements
- exception: **replaced elements**
 - embedded elements + `<option>` where width and height are configurable



CSS property `display: inline-block`

- Adjacent elements in one row
- **margin, padding, width, height** settable



CSS property `display: none`

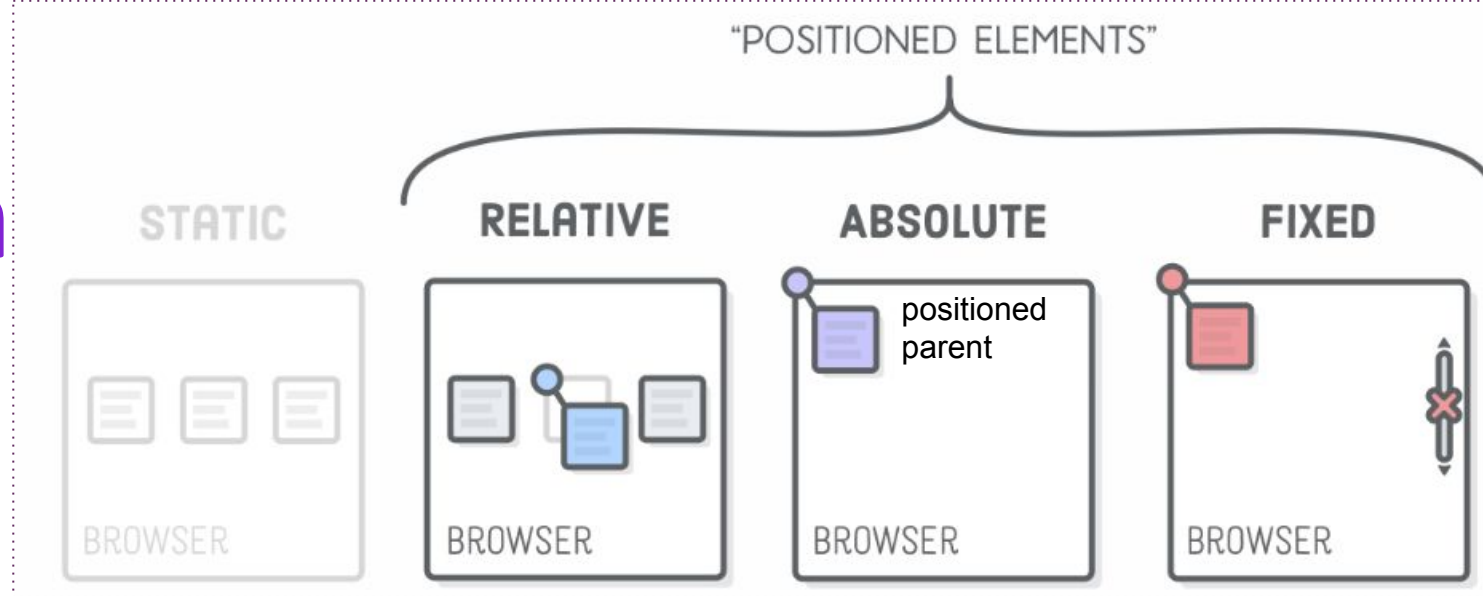
- o hides an element
 - the element is also taken out of the document flow
- o In addition to `none`, visibility can be controlled with **visibility** and **opacity**
 - element space remains occupied even if the element is not visible - in contrast to `display none` that clears the space

JWT

0243

display-esimerkki

Position



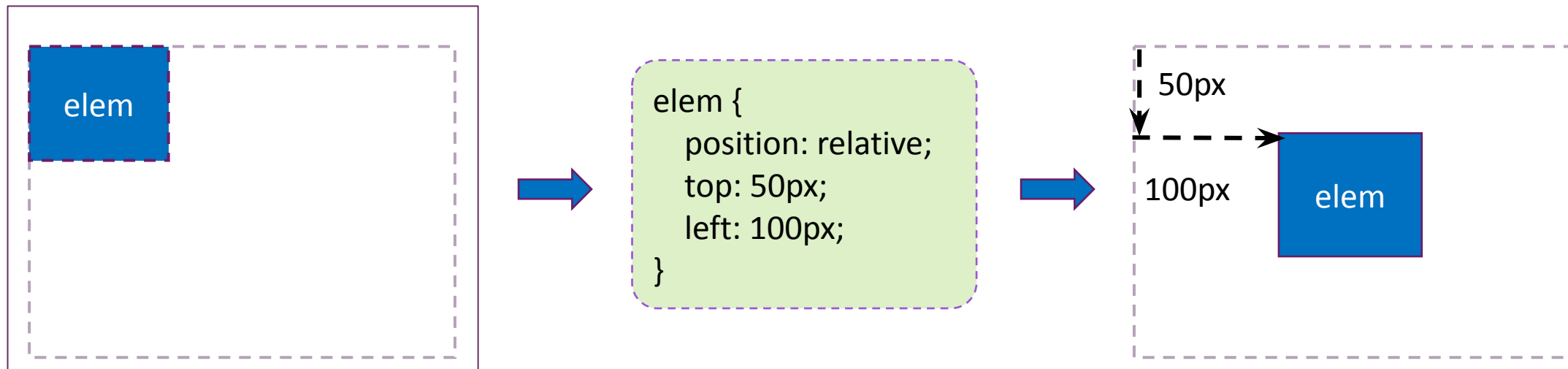
<https://www.internetingishard.com/html-and-css/advanced-positioning/>

- specifies how elements are positioned in a document
- default position is **static**
 - the normal position where the **document flow** puts the element
- others positioned in conjunction with properties
 - **top**, **right**, **bottom**, and **left**
 - e.g., **bottom**: **0px**;

Relative positioning

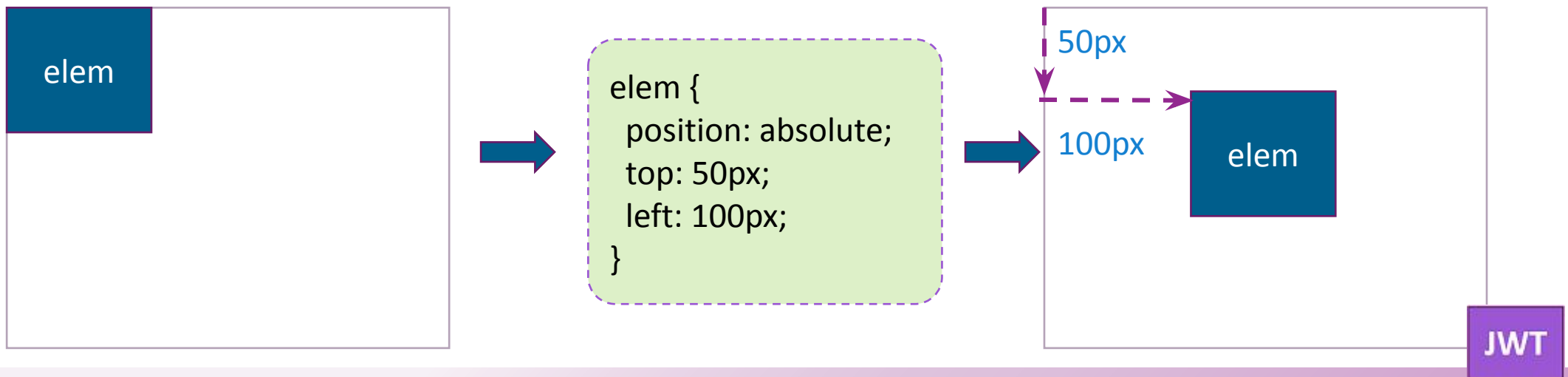
- moves an element **relative to its normal position**
 - set by properties `top`, `bottom`, `left`, `right`
(*from top, from bottom, from left, from right*)
 - element remains in the document flow
(the other elements assume it is in its original place)

JWT

0260
asemointi-relative

Absolute positioning

- Element is attached
 - to the closest ancestor whose position is other than static, and
 - taken out from the document flow



Fixed positioning

- Similar to absolute, but
 - not attached to an ancestor, but to the *viewport* (the browser window)



○ Sticky positioning

- a new, experimental positioning
 - kind-of-a combination of absolute and fixed positioning
 - fixed till a certain threshold, after that absolute



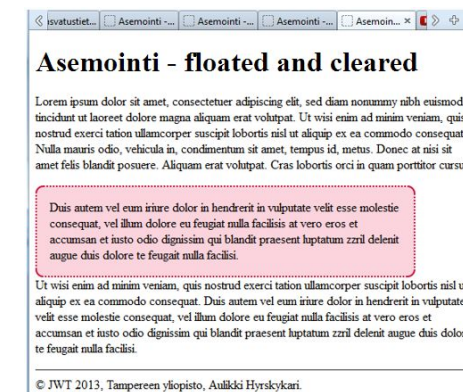
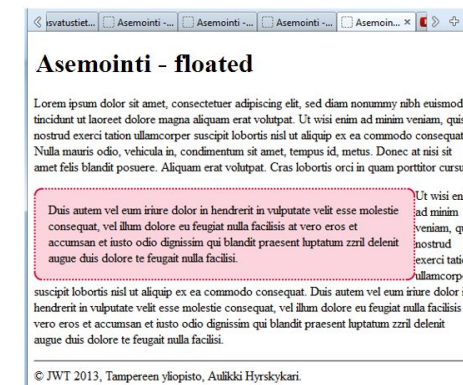
CSS float

float: none|left|right

- float sets element's horizontal position:
 - to the left or right edge of its parent
 - if scrolled, moves with its parent
 - removal from the document flow
 - following elements occupy the released space
 - the occupation is eliminated with clear
- clear: right|left|both
- A video, explaining it at a peaceful pace <https://youtu.be/fyWdofSeukE?t=51> (27 mins)

```
aside {
  float:left;
}
```

```
aside {
  float:left;
}
p.cleared {
  clear: left;
}
```



CSS Flexbox

`display: flex`

A Complete
Guide to
Flexbox
CSS-TRICKS

- Easier way to implement flexible layouts
- Elements are laid out within their parent: **flex container** whose display property is flex

```
<div class="container">  
  <!-- flex-items, the elements  
        which we want  
        to lay out -->  
</div>
```

```
div.container {  
  display: flex;  
}
```

- both horizontal and vertical directions are possible

Flex-directions row and column

flex-direction: row | row-reverse | column | column-reverse

```
<div class="container">  
  <div class="flex-item">Yyy</div>  
  <div class="flex-item">Kaa</div>  
  <div class="flex-item">Koo</div>  
</div>
```

```
div.container {  
  display: flex;  
  flex-direction: row-reverse;  
}
```



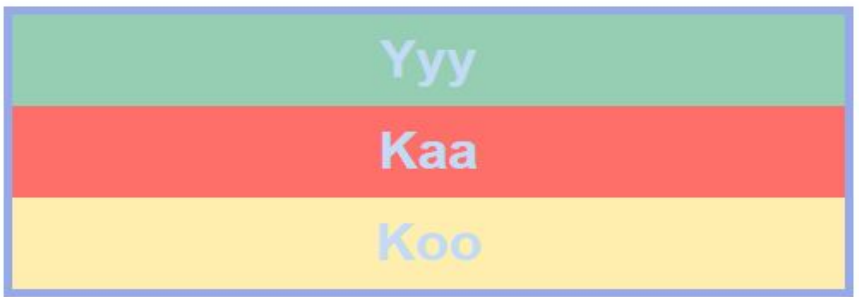
Koo Kaa Yyy

```
div.container {  
  display: flex;  
}
```



Yyy Kaa Koo

```
div.container {  
  display: flex;  
  flex-direction: column;  
}
```



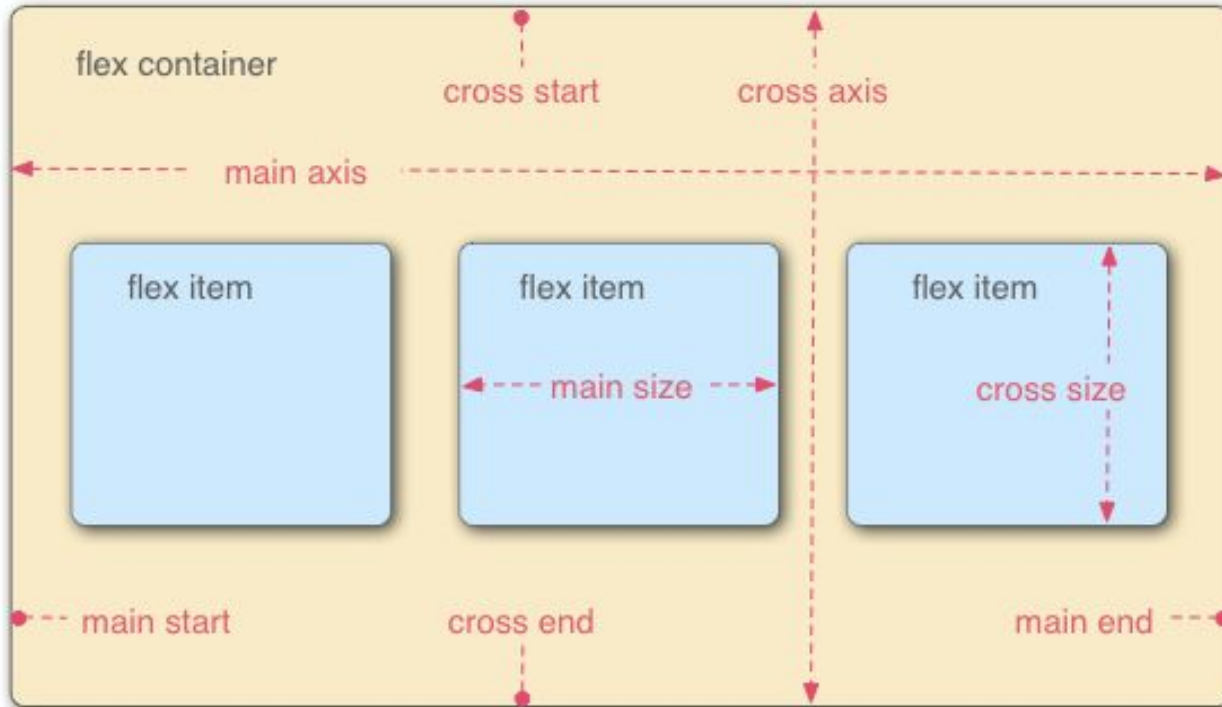
Yyy

Kaa

Koo

Flex-direction sets main- and cross axes

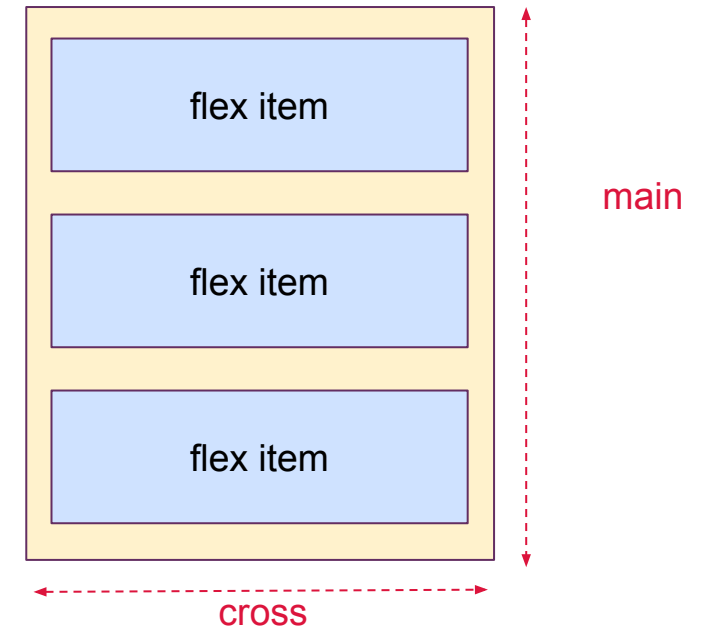
1



main and cross axes when flex-direction is

1. row
2. column

2



Justify-content along with main-axis

justify-content: flex-start | center | flex-end | space-around | space-between | ...

```
div.container {  
  display: flex;  
}
```

```
div.container {  
  display: flex;  
  justify-content: center;  
}
```

```
...  
justify-content: space-around;
```

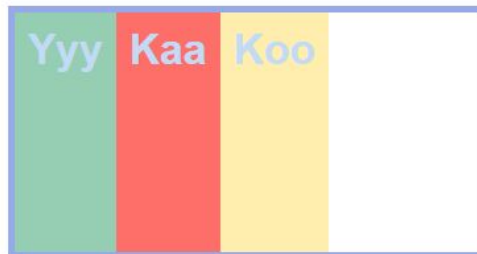
```
...  
justify-content: space-between;
```



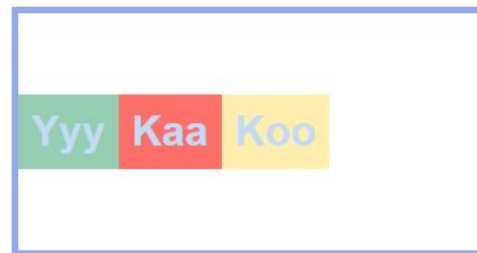
Align-items along with cross axis

`align-items: stretch | flex-start | flex-end | center | ...`

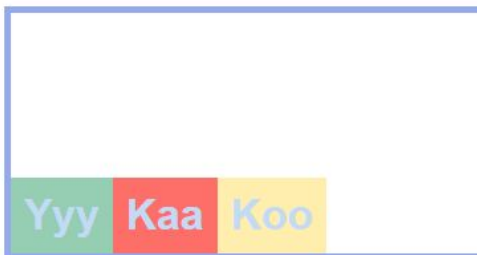
```
div.container {  
  display: flex;  
  height: 100%;  
}
```



```
div.container {  
  display: flex;  
  align-items: center;  
}
```



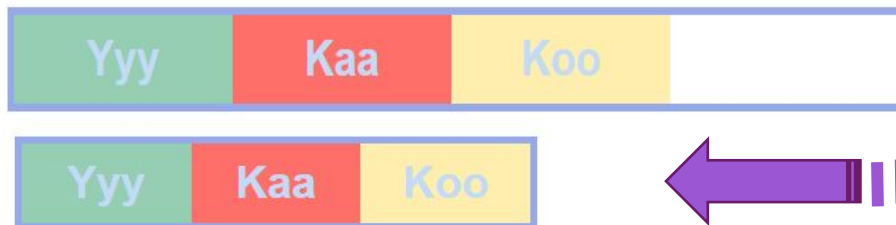
```
div.container {  
  display: flex;  
  align-items: flex-end;  
}
```



Multiline flex-container with flex-wrap

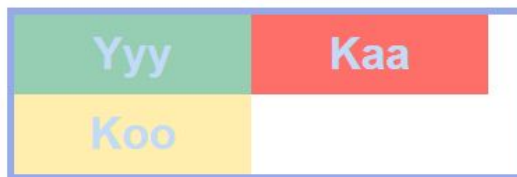
flex-wrap: nowrap | wrap |

```
div.container {  
  display: flex;  
}  
div.flex-item {  
  width: 150px  
}
```



- Multiline flex containers with flex-wrap

```
div.container {  
  display: flex;  
  flex-wrap: wrap;  
}  
div.flex-item {  
  width: 150px  
}
```



Flex item sizing declarations

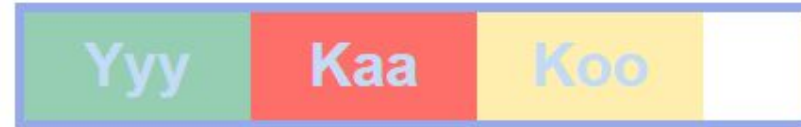
`'flex-grow' | ['flex-shrink'] | ['flex-basis']`

```
div.container {  
  display: flex;  
}  
div.flex-item {  
  flex: 1; /* = flex: 1 1 0, default */  
}
```

```
div.flex-item {  
  flex-basis: 150px;  
}
```



`flex-basis`
sets the initial size for flex-items;
the remaining space is left unused



Flex item grows

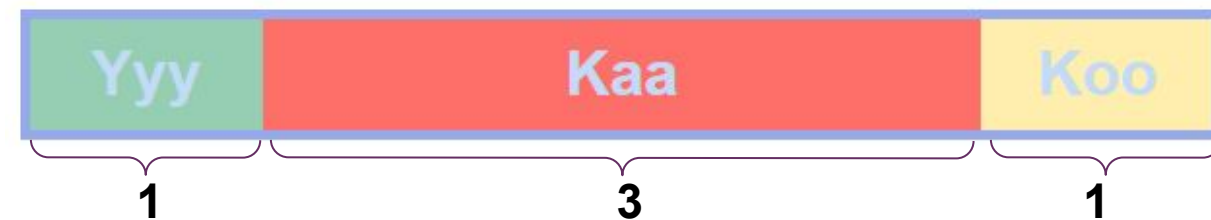
'flex-grow' | ['flex-shrink'] | ['flex-basis']

```
div.container {  
  display: flex;  
}  
div.flex-item {  
  flex-basis: 150px;  
}
```

```
div.flex-item:nth-child(2) {  
  flex-grow: 3;  
}
```



flex-grow
used to allocate the additional space for
flex-items (in relation to other items)



See the [Animated gif from freeCodeCamp](#):

Flex item shrinks

'flex-grow' | ['flex-shrink'] | ['flex-basis']

```
div.container {  
  display: flex;  
}  
div.flex-item {  
  flex-basis: 200px;  
}
```

```
div.flex-item:nth-child(2) {  
  flex-shrink: 0; /* = flex: 1 0 150px */  
                  /* = do NOT shrink! */  
}
```

```
div.flex-item:nth-child(2) {  
  flex-shrink: 5;  
}  
div.flex-item:nth-child(3) {  
  flex-shrink: 2;  
}
```



flex-shrink: 0
used to steal the decreasing size from
flex-items (in relation to each other)



[Example in Git](#)

Flex item

order: integer

```
div.container {  
  display: flex;  
}
```

```
div.flex-item:nth-child(2) {  
  order: 1;  
}  
div.flex-item:nth-child(3) {  
  order: -1;  
}
```



order
default is 0, the bigger number
the later in order



CSS Grid

display: grid

- CSS Grid can be used to layout entire web pages,
- CSS Grid for 2-D layouts vs. CSS Flexbox for 1-D layouts
- Defining CSS grid layout

```
<elem class= "container">  
    <!-- grid items -->  
</elem>
```

```
elem.container {  
    display: grid;  
    . . .  
}
```

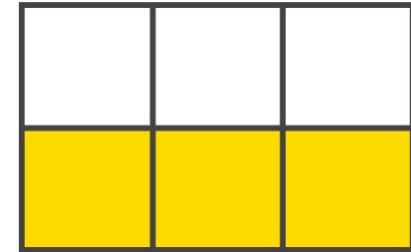
- **Grid container** contains **Grid items**

More terminology

- **Grid line**
row/column
grid lines



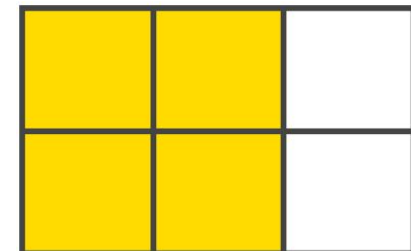
- **Grid tracks**
between
adjacent grid
lines



- **Grid cell**
between two
adjacent rows
and two adjacent
column grid lines



- **Grid areas**
surrounded by
four grid lines



Grid container: grid-template, grid-gap

grid-template: [CSS unit]* / [CSS unit]*

row heights

column widths

```
<main class= "container">
  <div>Solu 1</div>
  <div>Solu 2</div>
  <div>Solu 3</div>
  <div>Solu 4</div>
  <div>Solu 5</div>
  <div>Solu 6</div>
</main>
```

```
.container {
  display: grid;
  /* 2 x 3 grid */
  grid-template: 50px 50px / 150px auto 150px;
}
```

occupies remaining space



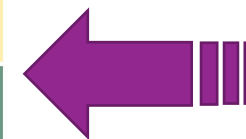
Grid container: new CSS unit, fraction (fr)

- width/3 (= 1 + 1 + 1): equal-width columns

```
.container {  
  grid-template: 50px 50px / 1fr 1fr 1fr;  
}
```

Solu 1	Solu 2	Solu 3
Solu 4	Solu 5	Solu 6

Solu	Solu	Solu
Solu	Solu	Solu



- width/10 (= 1 + 4 + 5): columns 1/10, 4/10 and 5/10

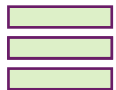
```
.container {  
  grid-template: 50px 50px / 1fr 4fr 5fr;  
}
```

Solu 1	Solu 2	Solu 3
Solu 4	Solu 5	Solu 6

Grid template shorthand: `repeat`

- o `repeat(nr, CSS unit value)`
equals **`nr` times repeated** CSS unit value, e.g.

```
grid-template: 50px 50px / 1fr 1fr 1fr;
```



```
grid-template: repeat(2, 50px)/ repeat (3, 1fr);
```

Grid areas: multiple grid cells

grid-column: start-value/end-value

grid-row: start-value/end-value

- **grid-column** and **grid-row** define multiple cells between specified grid lines:

```
.container :nth-child(3) {  
    grid-column: 1 / 4; /* from col 1 to 4, note: 4 not included */  
}
```

Grid area values

- The start and end values can be given as
 - grid-line numbers
 - `span n` defines the number rows or columns to span

```
.container :nth-child(3) {  
  grid-row: 2 / 3;  
  grid-column: 1 / 4;  
}
```

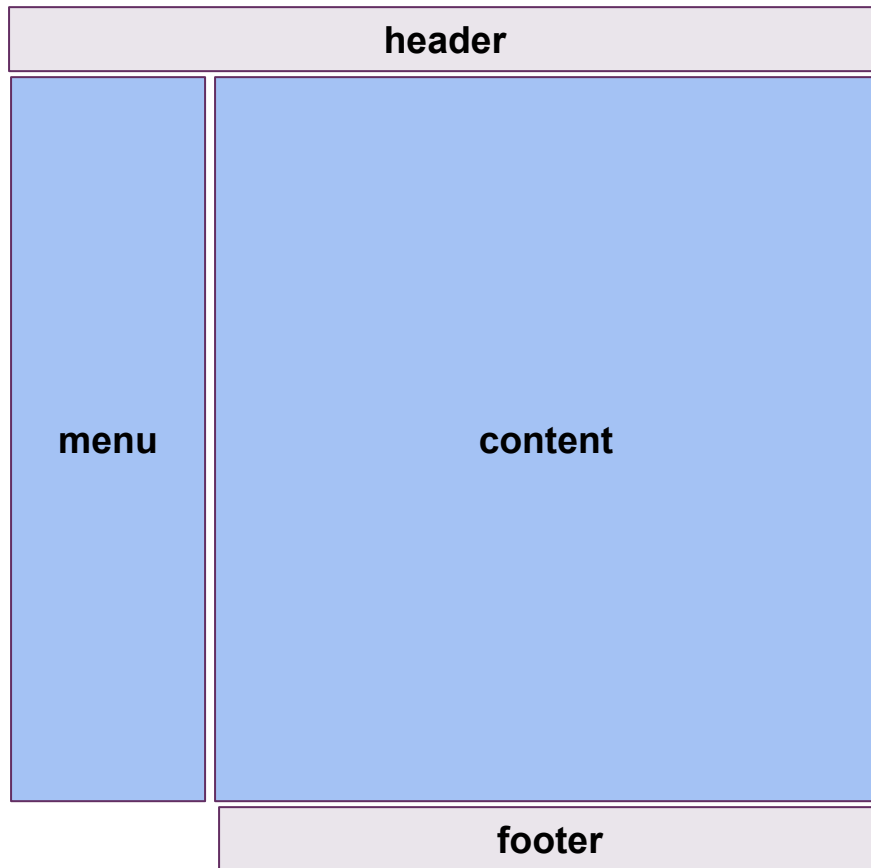


```
.container :nth-child(3) {  
  grid-row: 2 / span 1;  
  grid-column: 1 / span 3;  
}
```

- `auto`, automatic span, which indicates that it contributes nothing to the grid item's placement (auto-placement or a default span of 1)

grid-template-areas

- Grid can be defined by using grid-template-areas:

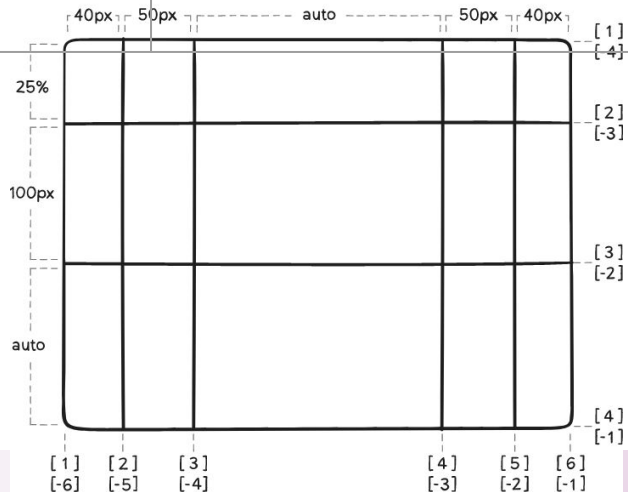


```
.container {  
  height: 100%;  
  display: grid;  
  grid-gap: 3px;  
  grid-template:  
    40px auto 40px / repeat(12, 1fr);  
  grid-template-areas:  
    "h h h h h h h h h h h h"  
    "m m c c c c c c c c c c"  
    ". . f f f f f f f f f f";  
}  
.header{ grid-area: h;}  
.menu{ grid-area: m;}  
.footer{grid-area: f;}
```

[“.” signifies an empty cell](#)

grid container

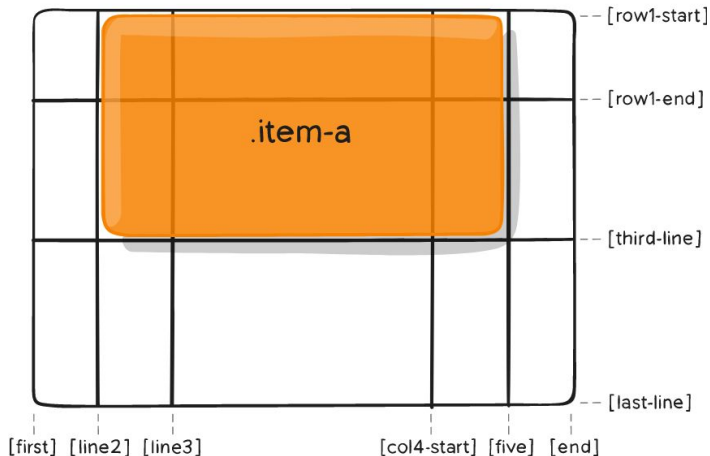
grid-template-columns	all prev. units + new fractions, e.g. <code>40px 50px auto 50px 40px;</code> auto occupies all space left
grid-template-rows	<code>25% 100px auto;</code>
grid-template	rows/columns <code>25% 100px auto/40px 50px auto 50px 40px;</code>
grid-template-areas	“ascii-art” presentation, e.g, “area1 area2” “area3 area3”;
column-gap, row-gap, gap	“margins”



grid items

grid-column-start/-end	grid-column-start: 2; grid-column-end: five;
grid-column	grid-column-start/-end, e.g., 2/five
grid-row-start/-end	grid-row-start: row1-start; grid-row-end: 3;
grid-row	grid-row-start/-end, e.g., row1-start/3
grid-area	row-start/column-start/row-end/column-end row1-start/2/3/five or name for grid-template-areas

```
.item-a {
  grid-column-start: 2;
  grid-column-end: five;
  grid-row-start: row1-start;
  grid-row-end: 3;
}
```



Order and tricks to disrupt grid flow

- HTML elements are laid out in the order of appearance
- By default, all grid items have an order of 0, but this can be set to any positive or negative value, similar to z-index.
 - if the main axis is from left to right
 - the smallest is on the left, the largest on the right
- It is also possible to span one grid item over the whole width:

```
.item {  
    grid-column: 1 / -1; /* -1 implies the last line */  
}
```
- if no specific handle, a handle to desired element may be got,
e.g., `.container :nth-child()`